UVM Introduction 2

Electrical and Information Technology, LTH Lund University

2026-09-28

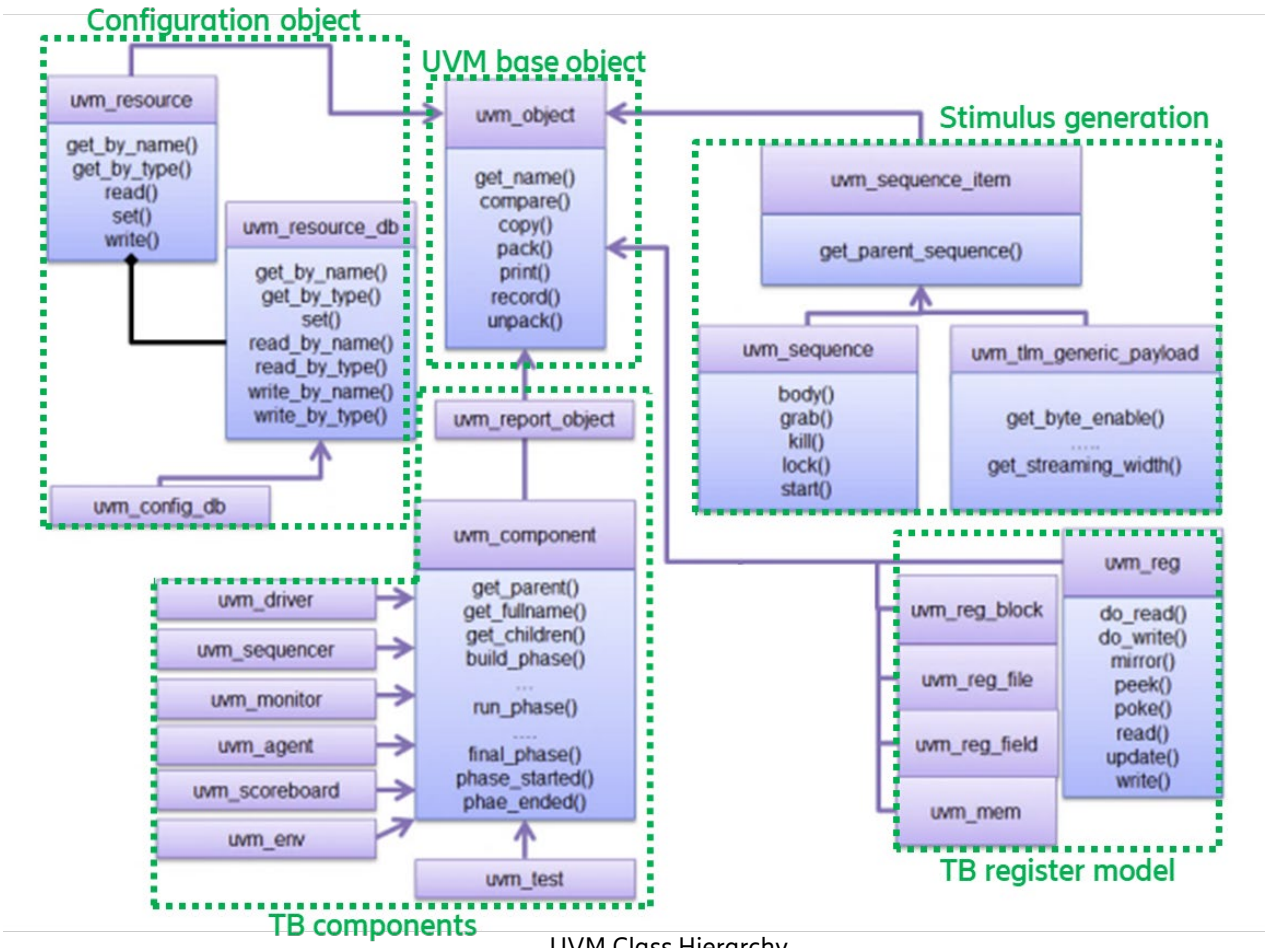
Ole Kristoffersen,
Senior Specialist Emerging Verification Technologies at Ericsson

Topics

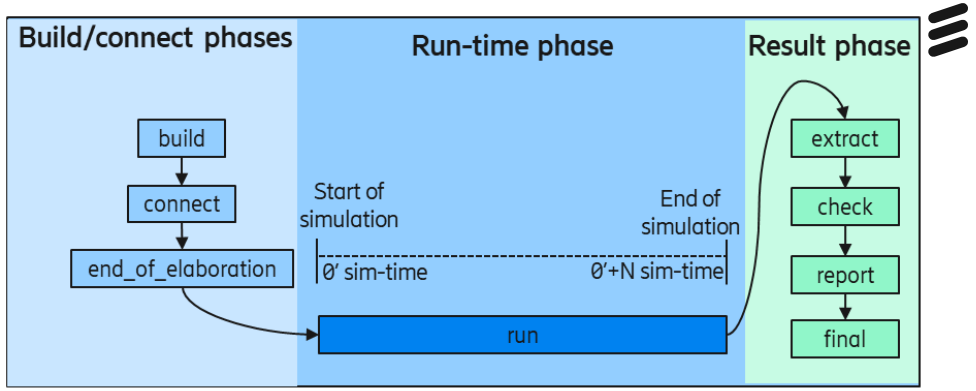
- UVM Recap
- UVM Basic Features
- UVM TB Infrastructure
- Coffee Break
- UVM Stimulus Generation
- Advanced UVM Features
- Create a UVM TB Step-by-Step
- Key Takeaways

UVM Recap

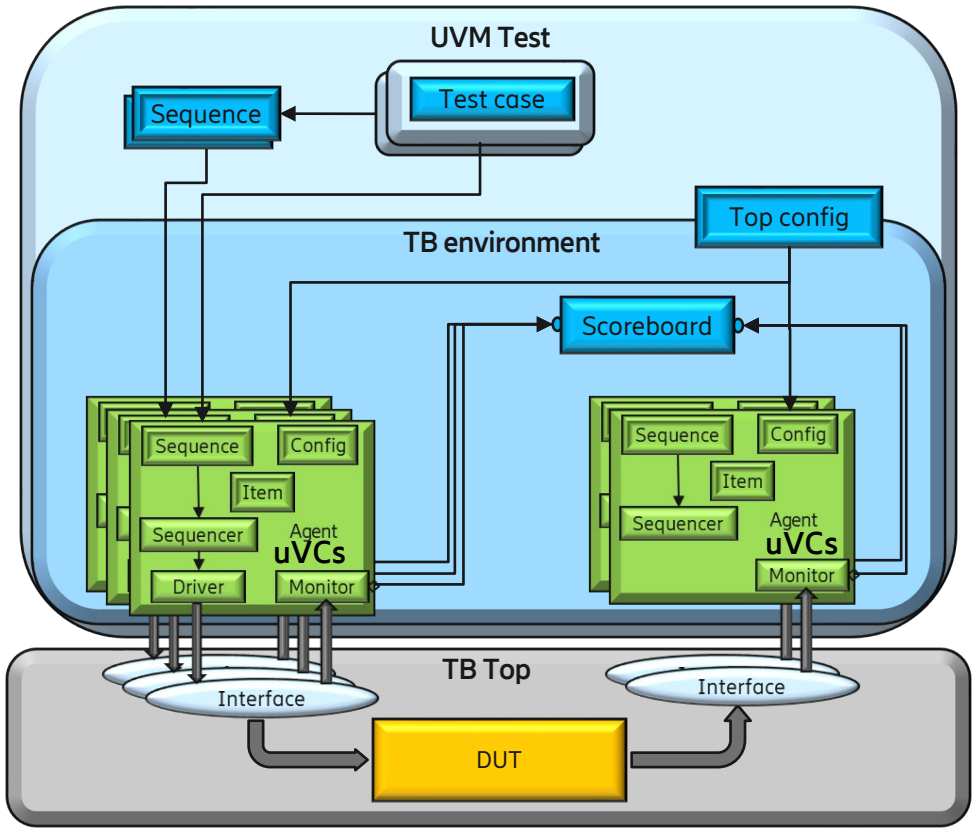
UVM Recap – Concept



UVM Class Hierarchy



UVM execution phases



Basic UVM architecture

UVM Recap – Summary

- As designs become more complex and larger, the verification challenge increases
- Creating a traditional SV TB becomes challenging when IP design is updated
- UVM standardizes TB framework using object-oriented methods for flexibility, reusability and scalability
- The UVM concept is based on well-defined components that are extended from UVM base classes
- UVM provides an easy way to both reuse UVM environments and scale the TB
- The UVM engineers' work can be divided in developing TB infrastructure and writing tests

UVM Basic Features

UVM Basic Features – UVM Report Macros

UVM info macro
- Provide a standardized way to report informational messages during simulation
- Support different verbose levels: NONE, LOW, MEDIUM, HIGH, FULL and DEBUG

UVM warning macro
- Report non-critical issues during simulation that do not halt the process but may indicate potential problems

UVM error macro
- Reports critical issues during simulation that indicate a failure or inconsistency in the design or TB

UVM fatal macro
- Reports critical issues that are so severe that they immediately stop the simulation

```
task my_task(int mode, int write, int expected);
  `uvm_info("my_task", "This is info message", UVM_MEDIUM)
  if (mode==0)
    `uvm_warning("my_task", "Mode is set to 0")
  if (write != expected)
    `uvm_error("my_task",$sformatf("Error write=%0d excepted=%0d",write,excepted))
  if (mode > 10)
    `uvm_fatal("my_task", "Simulation exit mode>10")
endtask : my_task
```

Example code with UVM massage macros

Simulator +UVM_VERBOSITY=UVM_<LEVEL>

```
UVM_INFO @ <sim_time> : <filename>(<line_number>) [my_task] This is info message
UVM_WARNING @ <sim_time> : <filename>(<line_number>) [my_task] Mode is set to 0
UVM_ERROR @ <sim_time> : <filename>(<line_number>) [my_task] Error write=12 excepted=14
UVM_FATAL @ <sim_time> : <filename>(<line_number>) [my_task] Simulation exit mode>10
```

Output in console and simulation log file

UVM Basic Features – Factory

UVM Factory

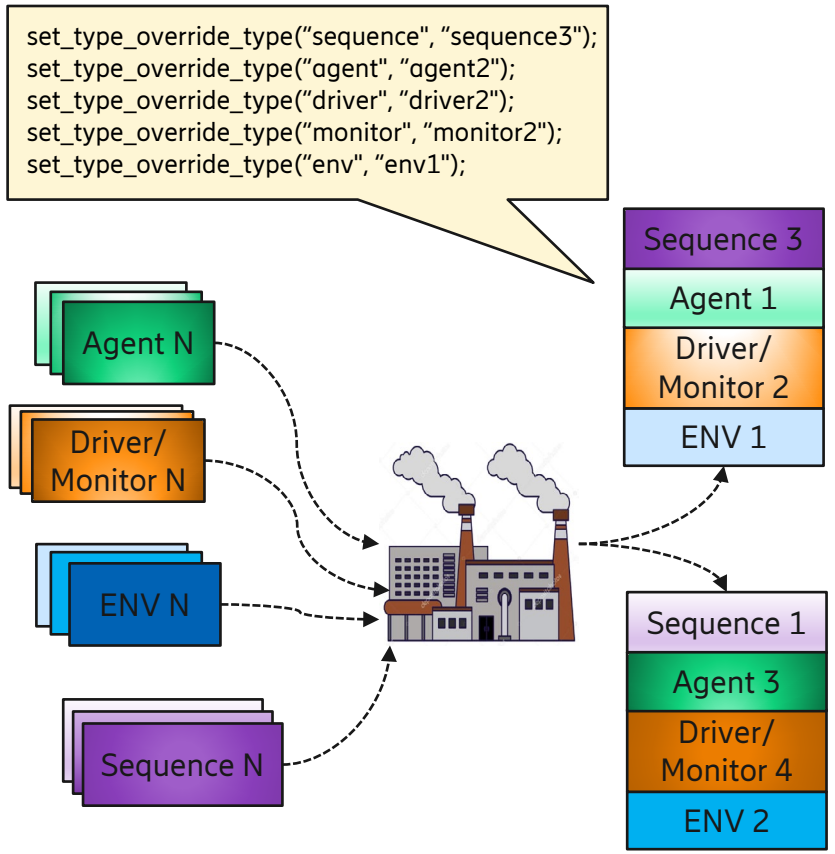
- Enables overriding base classes with derived ones without modifying the TB
- Supports parameterization and configuration of objects during simulation

All object should be registered with factory class

- Use `uvm_object_utils (for sequences)
  `uvm_component_utils (for components)
- Use ::create() method of factory instead of calling new()

Two types of override possible

- **Type Override:** Each time a component type is instantiated, a substitute type is created for all instances
- **Instance override:** A specific component instance is overridden by specifying its position in the UVM component hierarchy



A factory can create an object of the specified type

UVM Basic Features – TLM Connection

UVM TLM port connection between components

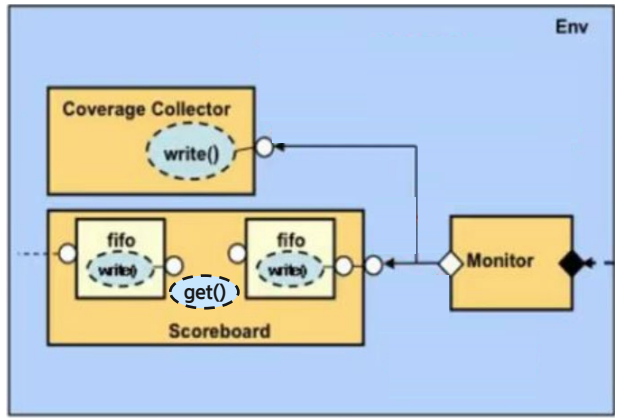
- Facilitates high-level communication between UVM components by transferring transactions
- TLM ports send transactions, exports receive them, and implementations define the functionality
- Supports blocking and non-blocking communication between components

Analysis Ports

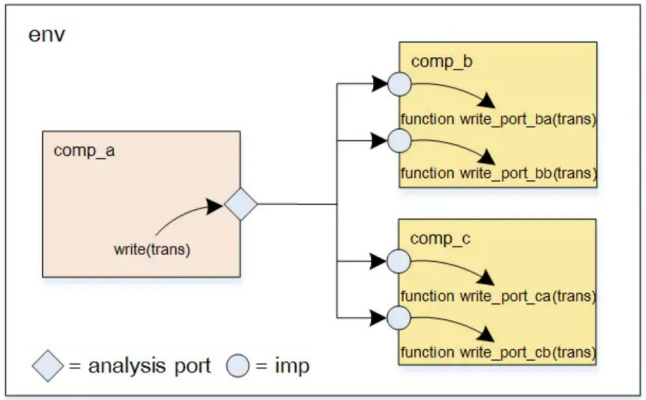
- The producer sends the same transaction to multiple subscribers using the write method

Analysis FIFO

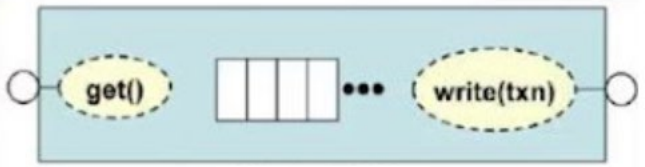
- Stores transactions in a first-in, first-out FIFO using the write method and retrieves data using the get method



Overview of TLM producer, analysis ports and analysis FIFO



Overview of TLM producer and analysis ports



Overview of a TLM analysis FIFO

UVM Basic Features – Config DB

Centralized parameter storage

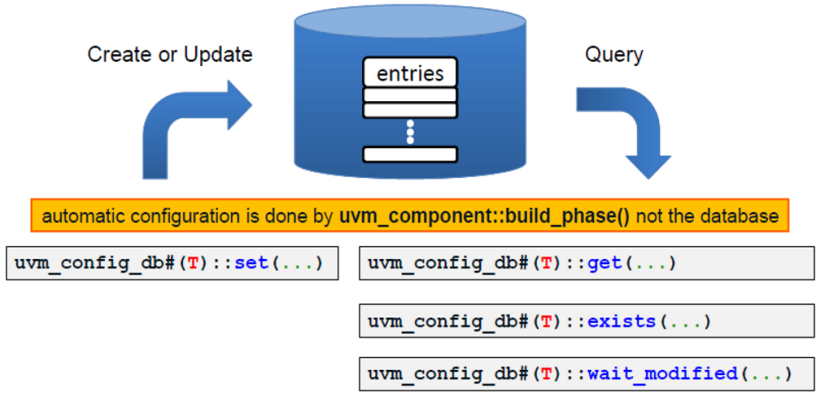
- Stores and manages configuration settings for the TB

Hierarchical access

- Allows components to access configurations from higher-level modules

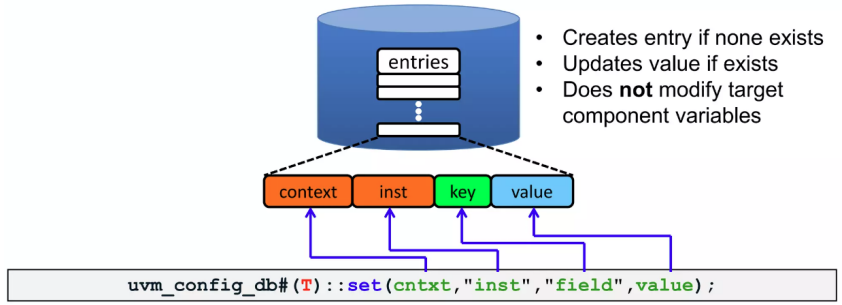
Flexible override

- Enables easy customization of specific components without changing the whole setup

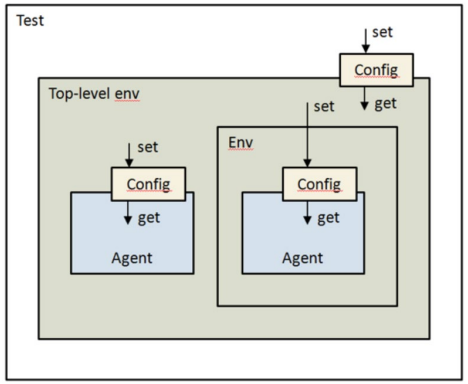


UVM config DB methods

Creating & Modifying Entries



UVM config DB method options



The top-level config is distributed to each component's config object

UVM TB Infrastructure

UVM TB Infrastructure – TB top

Key functionality of the TB top

- Instantiates the DUT, uVC interfaces, clocks, and resets
- Connect DUT signals with uVC interfaces signals including clock and reset
- Store virtual interfaces in UVM config DB to map a specific interface to a UVC
- Instantiates UVM and start test

Additional functionality

- Possible to instantiate other components if required for the TB
- Also instantiate assertions on DUT signals or internally within the DUT

```
module tb_top
 ...
 // Instantiate uVC interfaces
 reg_bus_if reg_bus_intf(clk,reset);
 // Instantiate DUT and connect uVC interfaces to the DUT
 my_dut my_dut_i(
   .clk(clk),
   .reset(reset),
   .addr(reg_bus_intf.addr),
   .wr_en(reg_bus_intf.wr_en),
   ...
 );
 // Store virtual interfaces in UVM config DB
 initial begin
    uvm_config_db#(virtual reg_bus_if)::
             set(uvm_root::get(),"tb_env.reg_bus_agent*","vif",reg_bus_intf);
    ...
 end
 // Instantiate UVM and start test
 initial begin
   run_test();
 end
endmodule : tb_top
```

Example of a typical UVM TB top

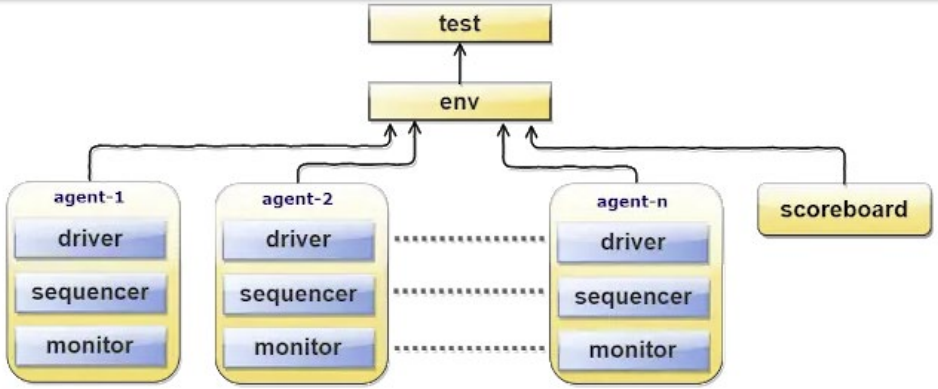
UVM TB Infrastructure – UVM Components

UVM components

- Used to create the TB infrastructure
- Once a component is created, it remains static throughout the entire simulation

UVM phases

- It is a sequence of well-defined steps that guide the TB construction, execution, and analysis of result



TB hierarchy for most used UVM components

```
class <my_class_name> extends uvm_<uvm_component>;
 `uvm_component_utils(<my_class_name>)
  ...
  function new (string name="<my_class_name>",uvm_component parent=null);
    super.new(name,parent);
  endfunction : new

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    // Add your build code
  endfunction : build_phase

  virtual function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    // Add your component connection code
  endfunction : connect_phase

  virtual function void end_of_elaboration_phase(uvm_phase phase);
    super.end_of_elaboration_phase(phase);
    // Add your code before it starts simulation
  endfunction : end_of_elaboration_phase

  virtual task run_phase(uvm_phase phase);
    super.run_phase(phase);
    // Add your simulation code
  endtask : run_phase

 virtual function void check_phase(uvm_phase phase);
    super.check_phase(phase);
    // Add your check code after simulation
  endfunction : check_phase

endclass : <my_class_name>
```

TB code template for UVM components

Phase name	Description
build	Used to construct the TB components
connect	Used to connect TLM ports of components
end_of_elaboration	Used to make any final adjustments to the structure, configuration or connectivity of the testbench before simulation starts
run	Used for stimulus generation, driving, monitoring, and checking
check	Used to check that the DUT behaved correctly and to identify any errors that may have occurred during the execution of the TB

Most used UVM phases in UVM components

UVM TB Infrastructure – UVM Test

UVM test main purpose

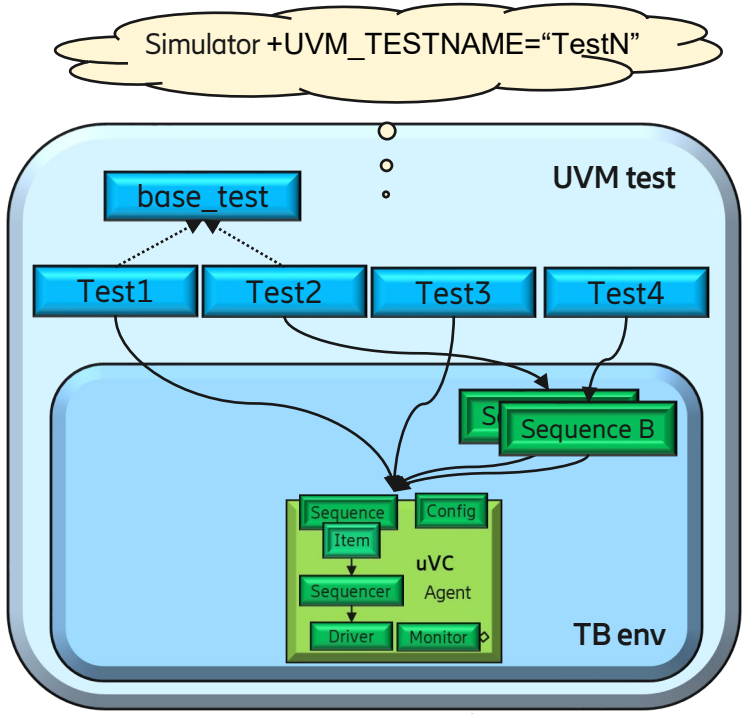
- Create/update TB Top configuration
- Create TB UVM environment
- Generate stimulus data for the DUT

UVM test implementation

- **Direct:** The test directly controls sequences and stimulus.
- **Inheritance:** Tests inherit from a base class, customizing behavior
- **Sequence-Based:** The test relies on reusable sequences for stimulus
- **Parameterized:** Tests are dynamically configured using the UVM factory or config database

UVM test selection

- Simulator argument +UVM_TESTNAME="<your_uvm_test_name>"



UVM test examples

```
class my_test extends uvm_test;
 `uvm_component_utils(my_test)
  ...
  virtual task run_phase(uvm_phase phase);
    super.run_phase(phase);
    // Must be added otherwise the simulation will not simulate
    phase.raise_objection(this);
    // Stimulus generation code
    // Must be added otherwise the simulation runs forever
    phase.drop_objection(this);
  endtask : run_phase

endclass : my_test
```

UVM test must contain phase raise and drop method calls

UVM TB Infrastructure – Environment

UVM environment main purpose

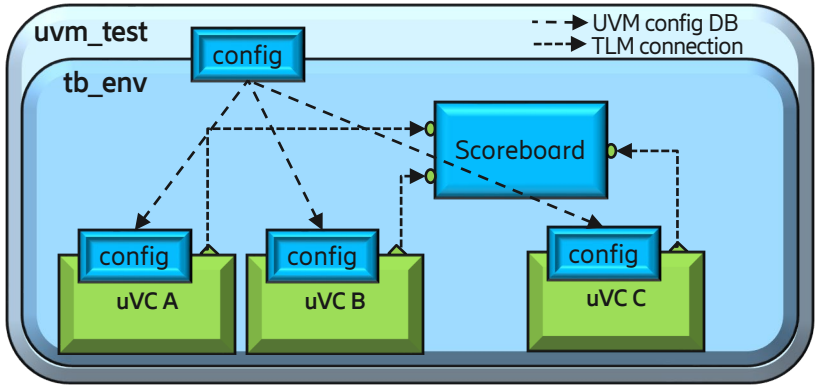
- Organizes and connects different verification components and uVC into a coherent TB structure

Infrastructure creation

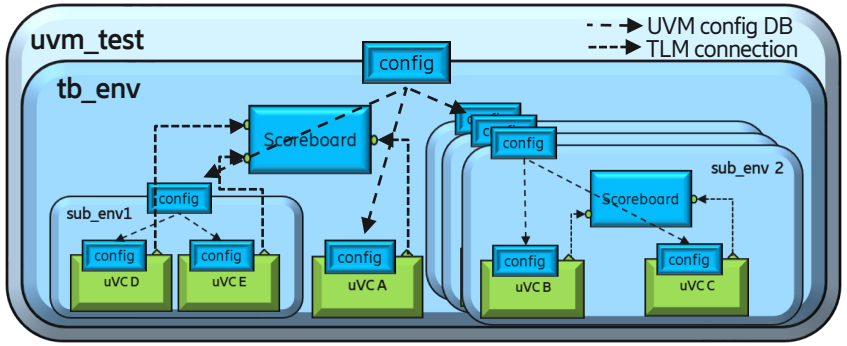
- Build phase
  - Create or distribute config objects into the UVM config DB
  - Create components in the environment
- Connect phase
  - Connect different components using TLM connections e.g. uVCs monitor to scoreboards

Hierarchical environments

- A TB can include multiple UVM environments in a hierarchical structure, enabling modularity and reusability

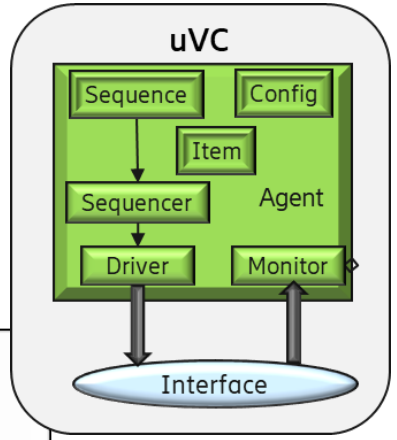


Single TB environment with all components instantiated



Complex TB environment with sub environments

UVM TB Infrastructure – Agent



Agent creates environment

- Get the agent config from UVM config DB
- Create sequencer, driver and monitor based on the config
- Create TLM connection between sequencer and driver

Config object

- Can contain any kind of configuration relevant to the uVC
- The virtual interface can be separated from the config object and placed in its own UVM config DB

Interface

- The driver controls the signals, while the monitor observes the interface signals

Agent extension

- A parameterized uVC interface can be supported
- Scoreboard and coverage collection can also be included
- Interface can contain tasks and assertions

```
class reg_bus_agent_cfg extends uvm_object;
  `uvm_object_utils(reg_bus_agent_cfg)
  // To set active/passive mode for the agent
  bit is_active;
  // Virtual interface
  virtual reg_bus_vif vif;
  ...
endclass : reg_bus_agent_cfg
```

Simple example of an agent configuration object

```
class reg_bus_agent extends uvm_agent;
  ...
  // Pointers to sequencer, driver, and monitor
  uvm_sequencer #(reg_bus_sequence_item) sequencer;
  reg_bus_driver       driver;
  reg_bus_monitor      monitor;
  reg_bus_agent_cfg cfg;
  ...
  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    // Get the config object
    if (!uvm_config_db#(reg_bus_agent_cfg)::get(this,"",
                                "reg_bus_agent_cfg",cfg)) begin
      `uvm_fatal(get_name(), "Config object not found")
    end
    // Create the monitor
    uvm_config_db#(reg_bus_agent_cfg)::set(this,"monitor","cfg",cfg);
    monitor = reg_bus_monitor::type_id::create("monitor", this);
    // Depending on config, create active or passive agent
    if (cfg.is_active) begin
      uvm_config_db#(reg_bus_agent_cfg)::set(this,"driver","cfg",cfg);
      driver   =reg_bus_driver::type_id::create("driver", this);
      sequencer=reg_bus_sequencer::type_id::create("sequencer", this);
    end
  endfunction : build_phase

  virtual function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    if (cfg.is_active) begin
      driver.seq_item_port.connect(sequencer.seq_item_export);
    end
  endfunction : connect_phase
endclass : reg_bus_agent
```

Simple example of an agent implementation

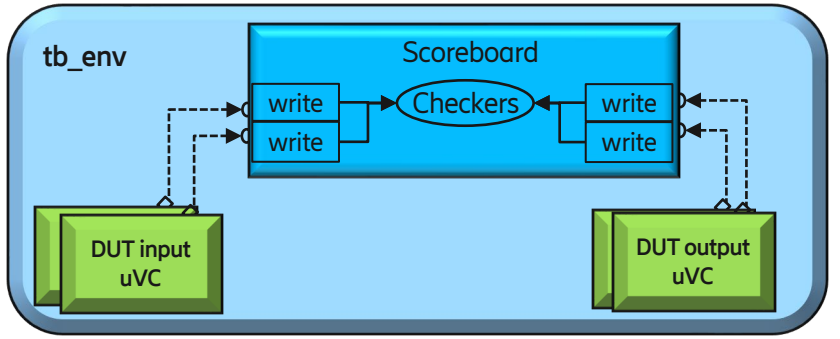
UVM TB Infrastructure – Scoreboard

Main feature

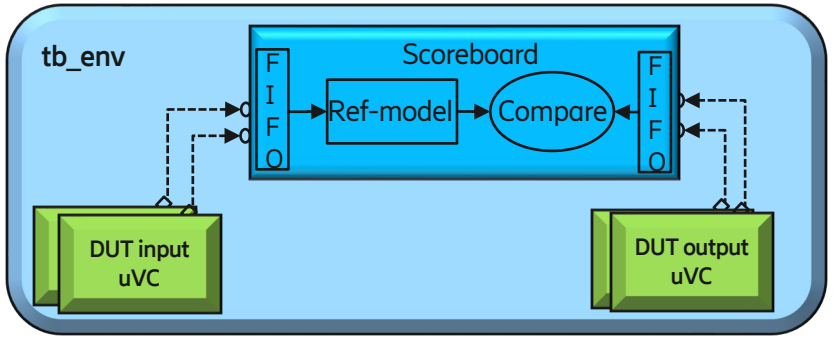
- Acts as a reference model to compare expected and actual DUT outputs
- Tracking transactions to validate behavior over time
- Receives transactions from the agent monitors via TLM connections

TLM connection

- Type of TLM connection depends on transaction type and complexity
- **Use analysis ports** for transactions that can be processed immediately, and the data arrives at a manageable rate
- **Use FIFO ports** to handle data bursts, manage varying transaction rates, or to ensure orderly processing of transactions
- Connection between uVC monitor and scoreboard is created in UVM environment connection phase method



Scoreboard with TLM analysis port write implementations



Scoreboard with TLM FIFO port implementations

```
// Instance analysis defines
`uvm_analysis_imp_decl(_scb_reset)
class scb extends uvm_scoreboard;
  `uvm_component_utils(scb)
  uvm_analysis_imp_scb_reset #(reset_seq_item, scb) reset_ap;
  uvm_tlm_analysis_fifo #(bus_seq_item) bus_fifo;
  ...
  virtual function void build_phase(uvm_phase phase);
    ...
    analysis_port = new("analysis_port", this);
    bus_fifo = new("bus_fifo", this);
  endfunction : build_phase
  virtual function void write_scb_reset(reset_seq_item reset_item);
    reset_checker(reset_item);
  endfunction : write_scb_reset
  virtual task run_phase(uvm_phase phase);
    bus_seq_item bus_item;
    forever begin
      bus_fifo.get(bus_item);
      bus_checker(bus_item);
    ...
  endtask : run_phase
endclass : scb
```

Example of scoreboard with both TLM analysis and FIFO ports

UVM TB Infrastructure – Coverage Collector (Subscriber)

Coverage collector

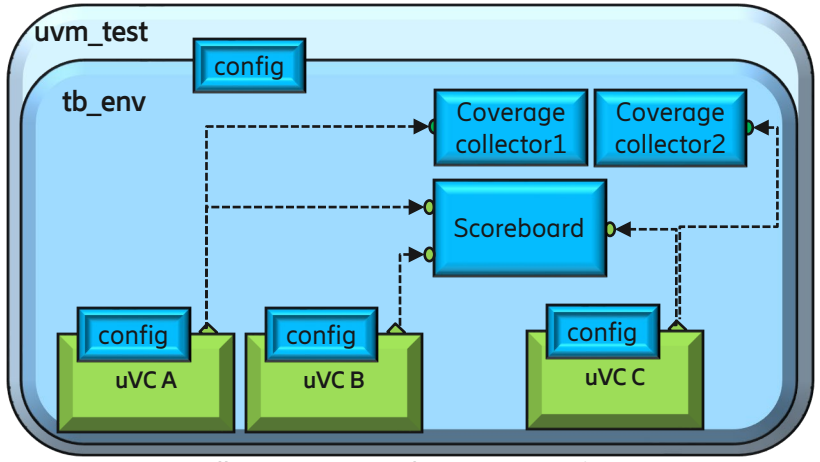
- Collect data from UVCs or scoreboards to generate and sample functional coverage

TLM connections

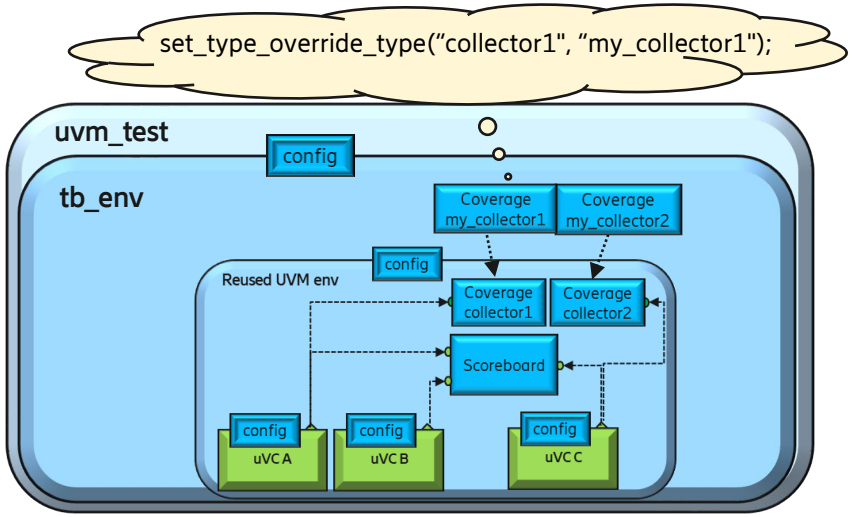
- Built-in TLM analysis port for receiving transactions from a connected analysis export
- Possible to add more TLM analysis ports as needed

Reusability at a higher level

- Higher-level reuse environment needs another level of functional coverage
- A higher-level TB can create new cover groups by extending the class and utilizing the UVM factory feature



Coverage collector connected to agent monitor TLM port



Example of using UVM factory to override coverage collector

Coffee Break

UVM Stimulus Generation

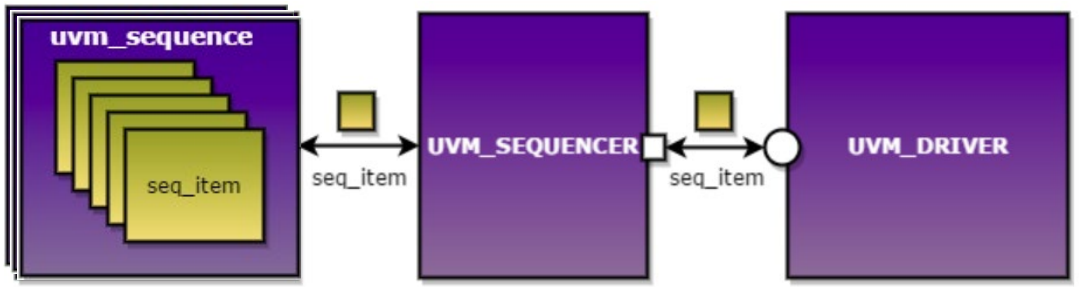
UVM Stimulus Generation – Communication between Sequence and uVC driver

Sequence
- It defines a series of transactions that will be sent to the Driver

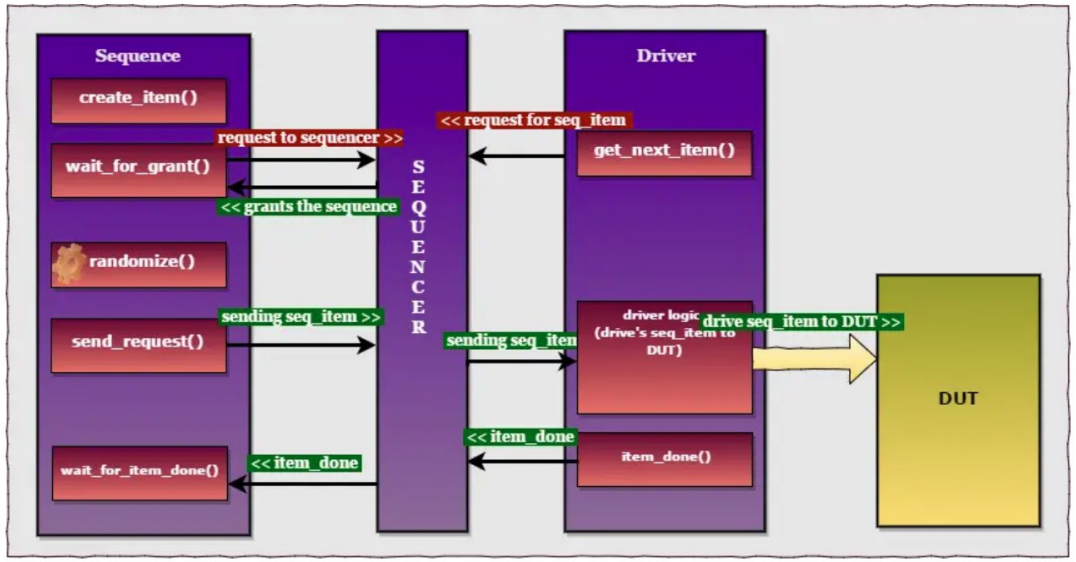
Sequence item
- This transaction object represents data or actions sent to the DUT, encapsulating the sequence's stimulus

Sequencer
- Responsible for managing the flow of the sequence and sending transaction items to the driver

Driver
- The driver converts these transaction items into pin-level signals for the DUT



Communication flow with sequence item



Communication handshake between sequence and driver

UVM Stimulus Generation – Sequence Item

Random data items
- Variables with constraints random values used by the uVC driver to drive the DUT signals

Useful function methods
- Specifies operations or routines within a class that act on data items

Field flag macros
- Controls field behaviors for various function methods

```
class bus_seq_item extends uvm_sequence_item;
  rand bit [6:0] addr;
  rand bit       wr_en;
  rand bit       rd_en;
  rand bit [7:0] wdata;
  // Read data
  bit [7:0] rdata;
  //constraints, to generate any one among write and read
  constraint wr_rd_c { wr_en != rd_en; };
  // Utility and Field macros,
  `uvm_object_utils_begin(bus_seq_item)
    `uvm_field_int(addr,UVM_ALL_ON)
    `uvm_field_int(wr_en,UVM_ALL_ON)
    `uvm_field_int(rd_en,UVM_ALL_ON)
    `uvm_field_int(wdata,UVM_ALL_ON)
    `uvm_field_int(rdata,UVM_ALL_ON)
  `uvm_object_utils_end
  // Constructor
  function new(string name = "bus_seq_item");
    super.new(name);
  endfunction
endclass : bus_seq_item
```

Example of sequence item code

Method name	Description
copy()	Copies data from one sequence item to another
clone()	Creates a deep copy of the sequence item
compare()	Compares two sequence items for equality
pack()	Packs the data into a bitstream for communication
unpack()	Unpacks data from a bitstream into the item
print()	Prints the contents of the sequence item for debugging

Useful function methods

FLAG name	Description
UVM_ALL_ON	Set all operations on (default)
UVM_DEFAULT	Use the default flag settings
UVM_NOCOPY	Do not copy this field
UVM_NOCOMPARE	Do not compare this field
UVM_NOPRINT	Do not print this field
UVM_NOPACK	Do not pack or unpack this field
....	

Some of the sequence item field flags

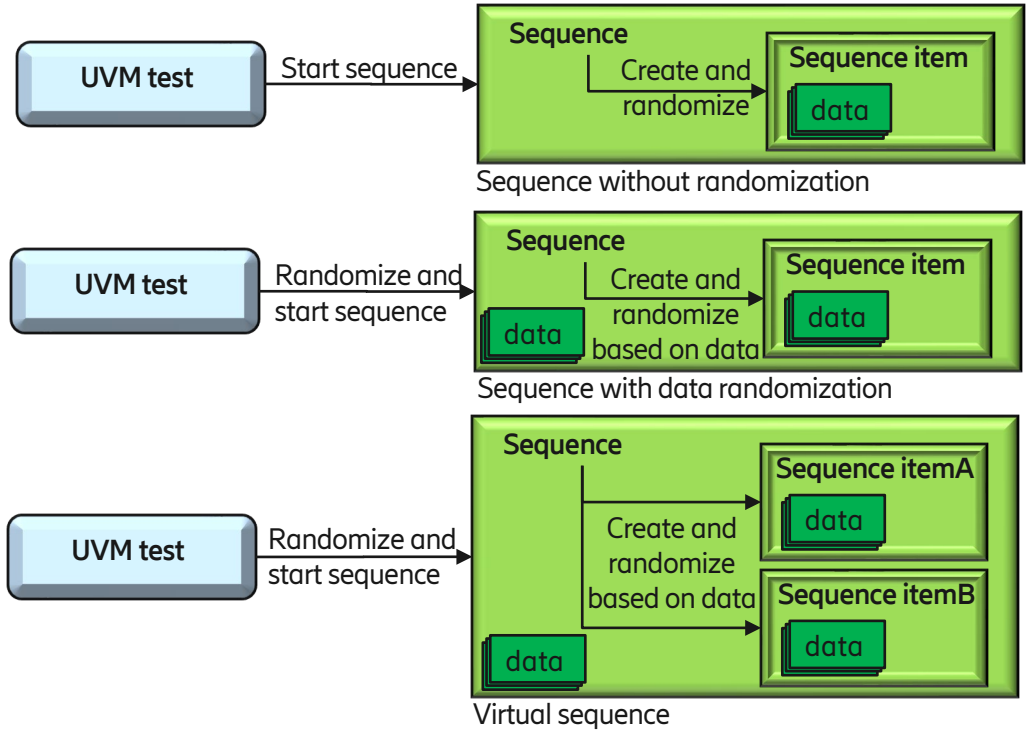
UVM Stimulus Generation – Sequence Use Models

Sequence usage models

The sequence itself is responsible for randomizing the sequence items

The sequence applies a runtime constraint to the transaction's data field to control the sequence item's value

Virtual sequence coordinates multiple sequences to drive complex test scenario on different uVC sequencers



UVM Stimulus Generation – Sequence Content

Sequence implementation

- Creates sequence item object and constrains random item values
- Handshake with the sequences to transfer the sequence item object

Sequence main method

- body() is main task that defines and runs transactions, sending items to the DUT via the sequencer and driver

```
class bus_test extends uvm_test;
  ...
  virtual task run_phase(uvm_phase phase);
    bus_access_seq bus_seq;
    super.run_phase(phase);
    phase.raise_objection(this);
    repeat(10) begin
      `uvm_rand_send_with(bus_seq, {
        wr == 1;
       })
    end
    ...
    phase.drop_objection(this);
  endtask : run_phase
endclass: bus_test
```

Example of UVM test using UVM macro to create, randomize and start a sequence

```
class bus_access_seq extends uvm_sequence#(bus_seq_item);
  `uvm_object_utils(bus_access_seq)
  rand bit [6:0] address;
  rand bit       wr;
  rand bit [7:0] data;
  //Constructor
  function new(string name = " bus_access_seq");
    super.new(name);
  endfunction : new

  virtual task body();
    //create the bus_seq_item
    req = bus_seq_item::type_id::create("req");
    wait_for_grant();        // wait for grant
    assert(req.randomize() with {
       addr == address;
       wr_en == wr;
       rd_en == ~wr;
       wdata == data;
    } ); // randomize the req
    send_request(req);       // send req to driver
    wait_for_item_done();    // wait for item done from driver
    get_response(rsp);       // get response from driver
  endtask : body
endclass: bus_access_seq
```

Example of sequence with randomization variables

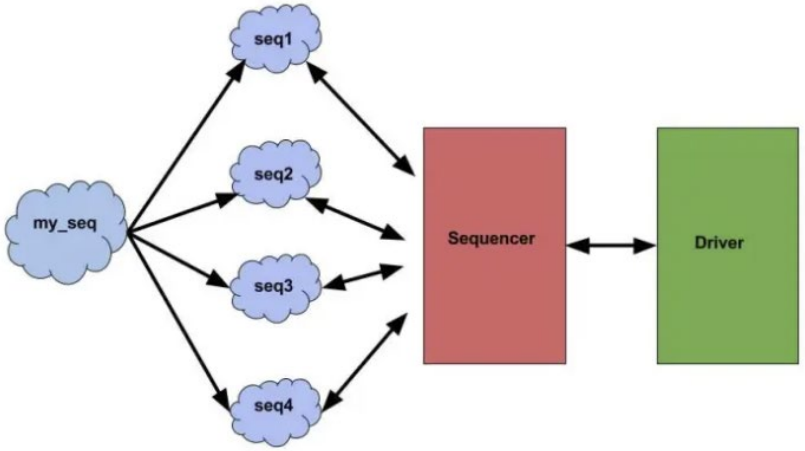
UVM Stimulus Generation – uVC Sequencer

Sequencer implementation

- The sequencer is a component that the uVC agent creates
- The standard sequencer is typically used, extension required only for advanced functions or protocol

Sequencer main purpose

- Acts as a middle layer, connecting the sequence to the driver
- Can arbitrate and prioritize between multiple sequences
- Coordinates when transactions are sent to the driver



Example of a use case with simultaneous multiple sequences

Method name	Description
set_arbitration()	Set arbitration: FIFO, Weighted, Random, Strict random, Strict FIFO or user specific
lock()/unlock()	Lock prevents other sequences from executing until the current sequence completes
grab()/ungrab()	Requests a lock for a specified sequence, which is granted when no other grabs or locks are blocking it

Sequencer function methods for arbitration and prioritization

UVM Stimulus Generation – uVC Driver

Driver implementation

- The driver is a component that the uVC agent creates
- uVC virtual interface is passed from the agent or UVM config DB
- The driver includes a TLM port for communication between the sequencer and the driver

Handling of item transaction

- The driver waits for any sequence item from the sequencer
- The driver drives the transaction to the DUT interface
- Response back to the sequencer when the transaction is complete

Method name	Description
get_next_item()	Blocks until a sequence item is available in the sequencer
try_next_item()	Attempts to fetch the next item without blocking, returns the sequence item if one is available in the sequencer
item_done()	Informs the sequencer that the current transaction is complete
put()	Sends an item back to the sequencer

Typical TLM port function methods used by the driver

```
class bus_monitor extends uvm_driver #(bus_seq_item);
  `uvm_component_utils(bus_monitor)
  ...

  virtual task run_phase(uvm_phase phase);
    forever begin
      seq_item_port.get_next_item(req);
       ......
       // Driving bus logic on the interface
       ......
      seq_item_port.item_done();
    end
  endtask : run_phase
endclass : bus_monitor
```

Example of uVC driver with highlights of communication to sequencer

UVM Checker – uVC Monitor

Passive signal monitoring

- uVC virtual interface is passed from the agent or UVM config DB
- UVM monitors passively observe DUT signals and convert them into transactions for analysis
- Monitors don't interfere with the DUT or alter its behavior

TLM analysis ports

- Monitors broadcast captured transactions via TLM analysis ports
- Transactions are sent to components like scoreboards and coverage collectors for further processing

```
class bus_monitor extends uvm_monitor;
  `uvm_component_utils(bus_monitor)
  uvm_analysis_port #(bus_seq_item) analysis_port;
  ...
  virtual function void build_phase(uvm_phase phase);
    ...
    analysis_port = new("analysis_port", this);
  endfunction : build_phase
  ...
  virtual task run_phase(uvm_phase phase);
    bus_seq_item seq_item;
    forever begin
      seq_item = bus_seq_item::type_id::create("seq_item");
      // Wait for next transaction by monitor bus logic on the interface
      ...
      seq_item.addr = ...
      ...
      // Broadcast transaction
      analysis_port.write(seq_item);
    end
  endtask : run_phase
endclass : bus_monitor
```

Example of uVC monitor with broadcast transaction to TLM analysis port


Advanced UVM Features

Advanced UVM Features – Register Model Architecture

Abstracts Hardware Registers

- Models DUT registers in a system, abstracting their address, fields, and behavior

Automates Read/Write

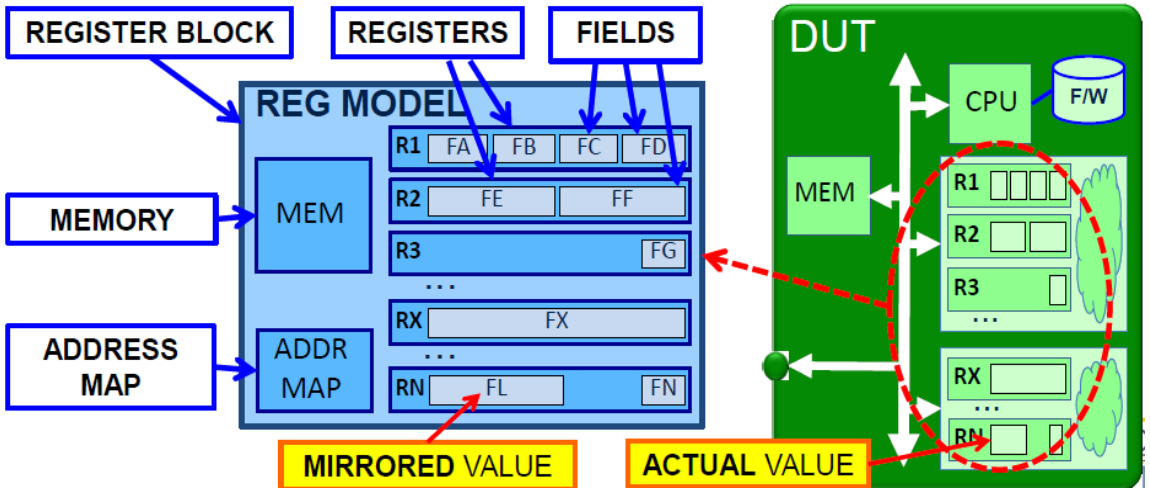
- Provides built-in methods for performing register read and write operations

Integrates with TB and a bus uVC

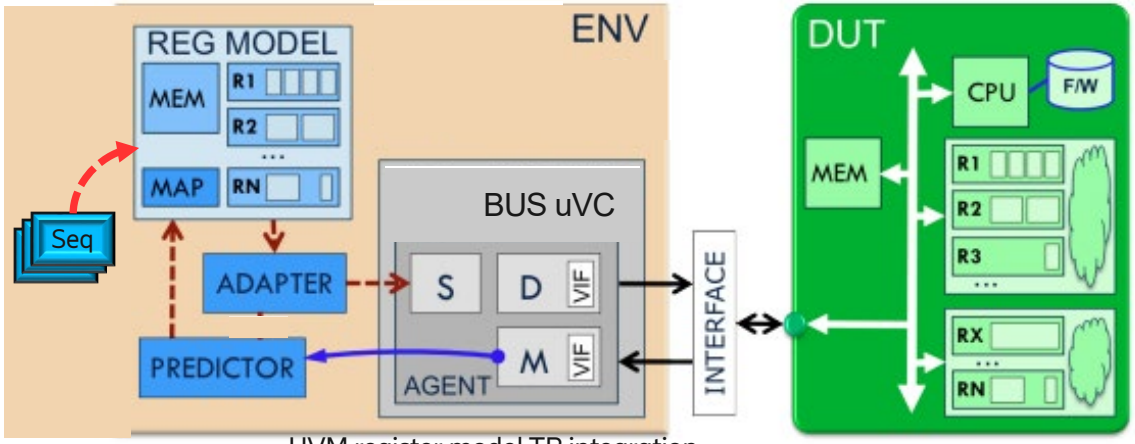
- Works with UVM sequences for automated and efficient register testing
- Connected to a bus uVC driver and monitor

Register model generation

- Typically, the UVM register model is automatically generated by a script



UVM register model



UVM register model TB integration

Advanced UVM Features – Register Model Usage

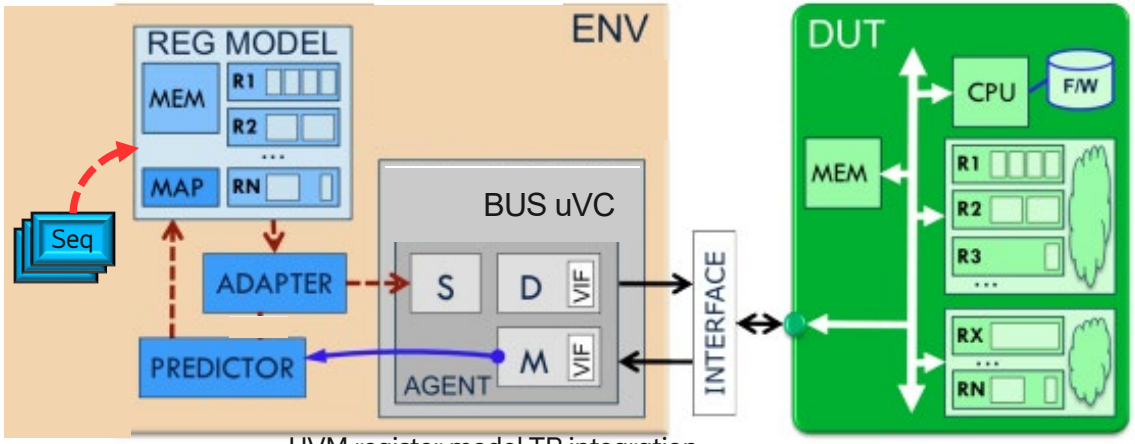
```
class register_check extends uvm_sequence
 `uvm_object_utils(register_check)
 reg_model my_regmodel;
...
 task body();
  uvm_status_e status;
  uvm_reg_data_t data;

  my_regmodel.R1.write(status, 255);
  if (status!=UVM_IS_OK) `uvm_fatal(get_name(), "Error during write R1")
  my_regmodel.R1.read(status, data);
  if (status!=UVM_IS_OK) `uvm_fatal(get_name(), "Error during read R1")
  // Set values in the fields in the register model
  my_regmodel.R1.field2.set(2);
  my_regmodel.R2.field3.set(1);
  // Updates DUT register with the register value in the register model
  my_regmodel.R2.update(status);
  if (status!=UVM_IS_OK) `uvm_fatal(get_name(), "Error during write R2")
  // Read the register back and check it against the value in the register model
  my_regmodel.R2.mirror(status, UVM_CHECK);
  if (status!=UVM_IS_OK) `uvm_fatal(get_name(), "Error during read R2")
 endtask : body
endclass: register_check
```

Example of accessing the registers from a sequence

Method name	Description
read()	Reads the current value from the HW register and UVM register
write()	Writes a value to both the HW register and the UVM register
update()	Writes the updated value stored in the UVM registers to the HW register
mirror()	Reads the value from the HW register and updates the UVM register
predict()	Updates the UVM register with an expected value
set()	Directly sets a value in the UVM register
get()	Retrieves the value from the UVM register

List of most used register model methods



UVM register model TB integration

Advanced UVM Features – Parameterized Interface In an uVC

Required uVC updates

- All components in the UVC must be implemented as parameterized classes
- uVC can be instantiated with different parameter sets in the TB environment

Benefits of parameterized classes in UVC

- Parameterized classes enables a flexible and reusable UVC for various DUT configurations
- This enhances the modularity and maintainability of the verification environment

Drawbacks

- Increases complexity by requiring careful parameter management across the UVM TB
- For sequencer, sequence and item it is possible to avoid parameterization by using a fixed max bit size

```
class tb_env extends uvm_environment;
  ...
  parameter ADDR_WIDTH = 4;
  reg_bus_agent #(ADDR_WIDTH) reg_bus;
  ...
  virtual function void build_phase(uvm_phase phase);
    ...
    reg_bus = reg_bus_agent #(ADDR_WIDTH)::type_id::create("reg_bus", this);
  ...
class reg_bus_agent #(parameter ADDR_WIDTH=8) extends uvm_agent;
  `uvm_component_param_utils(reg_bus_agent #(ADDR_WIDTH))
  ...
  reg_bus_driver #(ADDR_WIDTH) driver;
  reg_bus_monitor #(ADDR_WIDTH) monitor;
  ...
  virtual function void build_phase(uvm_phase phase);
    ...
    driver = reg_bus_driver #(ADDR_WIDTH)::type_id::create("driver", this);
    monitor= reg_bus_monitor #(ADDR_WIDTH)::type_id::create("monitor", this);
  endfunction : build_phase
  ...

class reg_bus_driver #(parameter ADDR_WIDTH=8) extends uvm_driver;
  `uvm_component_param_utils(reg_bus_driver #(ADDR_WIDTH))


  ...
  reg_bus_if #(ADDR_WIDTH) vif;
  ...

interface reg_bus_if #(parameter ADDR_WIDTH=8) (logic clk, logic rst_n);
  bit [ADDR_WIDTH-1:0] addr;
  bit       wr_en;
  bit       rd_en;
  bit [7:0] wdata;
  bit [7:0] rdata;
  ...
```

Example of an agent with parameterized components

Create a UVM TB Step-by-Step

Create a UVM TB Step-by-Step – Hello World Test

TB top module file

- Add start function to start UVM test

UVM test file

- Implement build phase function with create TB environment
- Implement run phase function with a simulation time consumption and add a UVM massage

TB environment file

- Implement build and connect phase functions with UVM messages

TB package file

- Create package with UVM and TB files

Get it to work

- Compile and simulate

```
module tb_top;
  // Import from UVM package
  import uvm_pkg::*;
  // Start UVM test_base environment
  initial begin
    run_test("hello_world_test");
  end
endmodule
```

tb_top.sv file

```
`timescale 1ns/1ns
package tb_pkg;
  // Import from UVM package
  import uvm_pkg::*;
  `include "uvm_macros.svh"
  // Import TB UVM files
  `include "tb_env.svh"
  `include "hello_world_test.svh"
endpackage: tb_pkg
```

tb_pkg.sv file

```
class hello_world_test extends uvm_test;
    `uvm_component_utils(hello_world_test)
    tb_environment tb_env;
    function new (string name="hello_world_test", uvm_component parent=null);
        super.new(name,parent);
    endfunction : new
    virtual function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        // Create UVM TB environment
        tb_env = tb_environment::type_id::create("tb_env", this);
    endfunction : build_phase
    virtual task run_phase(uvm_phase phase);
        super.run_phase(phase);
        // Raise objection if no UVM test is running
        phase.raise_objection(this);
        #100ns;
        `uvm_info(get_name(), "UVM Hello World test passed", UVM_LOW)
        #100ns;
        // Drop objection if no UVM test is running
        phase.drop_objection(this);
    endtask : run_phase
endclass : hello_world_test
```

hello_world.svh file

```
class tb_env extends uvm_environment;
    `uvm_component_utils(tb_env)

    function new (string name="tb_env", uvm_component parent=null);
        super.new(name,parent);
    endfunction : new
    virtual function void build_phase(uvm_phase phase);
      super.build_phase(phase);
      `uvm_info(get_name(), "TB environment built", UVM_HIGH)
    endfunction : build_phase
    virtual function void connect_phase(uvm_phase phase);
      super.run_phase(phase);
      `uvm_info(get_name(), "TB environment connected", UVM_HIGH)
    endfunction : connect_phase
endclass : tb_env
```

tb_env.svh file

Create a UVM TB Step-by-Step – Implement uVC template

Create and implement template files for the different uVC parts

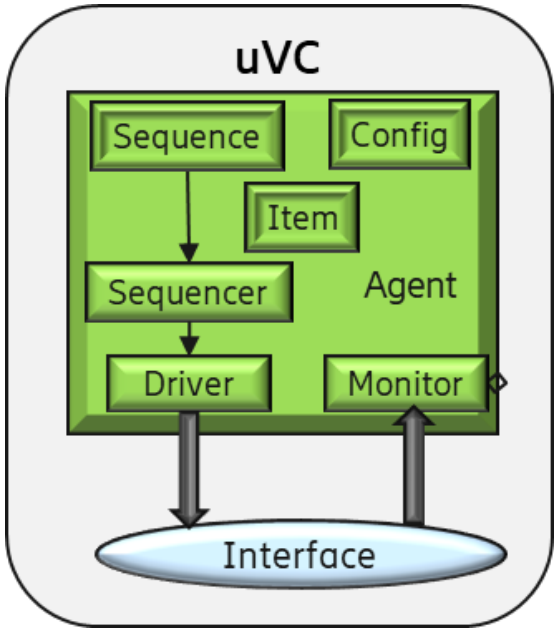
- Agent, driver and monitor components and config, sequence and item objects and final uVC interface

Integrate the uVC template into the TB

- Create uVC config, save in UVM config DB and instantiate uVC in the TB environment
- Update TB top file with uVC interface and store virtual interface in config DB

Compile and run hello world test

- This verifies that everything functions together
- Ensure the configuration and interface are properly mapped to the uVC



Create a UVM TB Step-by-Step – Finalize uVC and TB implementation

Upgrade uVC with specific functionality

- First start with upgrade config, item, sequence, driver and interface to enable stimulus generation to drive DUT signals

Implement UVM tests

- Create UVM tests and sequences to generate the required test stimulus for the DUT

Update TB with scoreboard and coverage

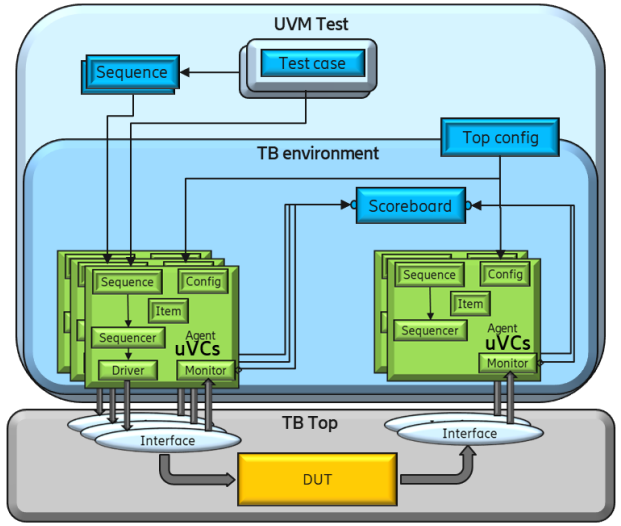
- Upgrade uVC with monitor the functionality
- In the TB environment, implement the scoreboard and coverage collectors, and connect TLM ports to the agent monitors

Always compile and run a test after making small set of updates

- This simplifies debugging and fixing code or implementation mistakes

Continuous implementation

- Continue to implement uVC and TB functionally step by step until completed



Basic UVM architecture

Create a UVM TB Step-by-Step – With GenAI (ChatGPT)

Always let GenAI create the UVM testbench step by step

- First, prompt it to generate the template files, then add features incrementally
- The more details provided, the more accurate the generated output will be

Review each result and provide corrections as needed

- Ask if there are any issues/limitations and whether a better solution exists

Save all prompts and use them, along with the source files, as context when starting a new prompt

- This helps the model understand your intentions and background

GenAI is excellent at analyzing problems and fixing them in RTL and TB code

- Add the error message or incorrect behavior, along with the expected outcome

Remember that you are responsible for the code, not GenAI model


Key Takeaways

Key Takeaways

- UVM Config DB is a key feature for distributing configuration across different hierarchical levels
- The factory feature allows for swapping components or objects without modifying the original code
- UVM components and uVCs built the infrastructure in the TB
- The UVM sequence and items are the primary object used to generate stimulus data
- The UVM register model should be used when DUT contains registers
- The uVC is the central component that translates transactions to and from pin toggling on DUT signals
- A parameterized interface in a uVC adds complexity but improves flexibility and reusability
- When creating a UVM TB from scratch, design the architecture first and then implement it step by step
- UVM supports multiple implementation styles, but they serve the same purpose
- GenAI changes the way of working, shifting the focus from coding to solving problems

Q & A

Email: ole.kristoffersen@ericsson.com

Links

- References
  - UVM (chipverify.com)
  - UVM Introduction (verificationguide.com)
  - Universal Verification Methodology (learnuvmverification.com)
  - UVM API 1.2 (verificationacademy.com)
  - Verification Methodology Cookbooks (verificationacademy.com)
  - uvm guide book 1.2 (accellera.org)
  - Easier UVM (doulos.com)
  - Easier UVM Code Generator Reference Guide (doulos.com)
  - UVM 1800.2-2020 IEEE Language Reference Manual (IEEE Xplore)
  - UVM DAC 2011 (slideshare.net)
  - UVM TUTORIAL DvCon2014 (slideshare.net)

